

Programming Project: Combinational Circuit Chip

Instructions: Read through the project report and submit all answers to the red sections and your source files in the 'Projects' directory of your GitHub repository.

Introduction

1. Learning Objectives

Upon completing the project, you will be able to:

- Explain what a combinational circuit is.
- Construct circuits from truth tables.
- Simplify a circuit using Boolean algebra and Karnaugh map.
- Combine simple circuits into a complex circuit.

2. Problem Statement

Can a 4-bit arithmetic logic unit that performs **only** addition and subtraction be constructed as a combinational circuit?

3. Scope & Constraints

Constraint	Details
Implementation Language	C++
Allowed Libraries	'iostream', 'string', 'sstream', 'fstream', 'iomanip', 'cctype'
Required Header Files	'Object.h', 'Dev.h', 'Gate.h', 'Split.h', 'Word.h', 'Port.h', 'Socket.h'
Helper & Sample Files	'sample.cpp'
Submission Deadline	Friday, May 8 - end of day

4. Submission Requirements

Item	Format
Completed lab report (this document)	.pdf
Source code files	.cpp, .h
Outputs	.txt

Report

1. Key Terms & Definitions

Define the following terms in your own words.

- Boolean Algebra
- Gate
- Combinational Circuit
- Truth Table
- Karnaugh Map

2. Reiterate Question

Restate the problem statement in your own words.

3. Forecasting

Before writing any code, describe what you already know by completing the following work.

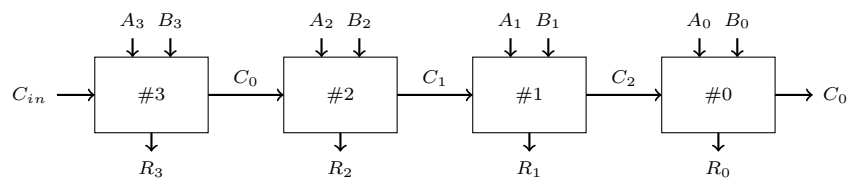
- a) State, in detail and in your own words, the relationship between addition and subtraction operations of two's-complement n-bit words.
- b) Complete the truth table by providing the expected outcomes given that R is the resultant of $A + B + C_{in}$ with C_{in} being the current carry, and C is the resultant carry of their sum.

A	B	C _{in}	R	C
1	1	1		
1	1	0		
1	0	1		
1	0	0		
0	1	1		
0	1	0		
0	0	1		
0	0	0		

- c) Write a Sum of Product (SOP) Boolean function for both R and C ,
d) Simplify each Boolean function using Boolean algebra postulates below and/or a Karnaugh Map.

Postulates		
$A \cdot B = B \cdot A$	$A + B = B + A$	Commutative Law
$(A \cdot B) \cdot C = A \cdot (B \cdot C)$	$(A + B) + C = A + (B + C)$	Associative Law
$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$	$A + (B \cdot C) = (A + B) \cdot (A + C)$	Distributive Law
$1 \cdot A = A$	$0 + A = A$	Identity Property
$A \cdot A' = 0$	$A + A' = 1$	Inverse Property
$A \cdot A = A$	$A + A = A$	Idempotent Property
$0 \cdot A = 0$	$1 + A = 1$	Annulment Property
$(A \cdot B)' = A' + B'$	$(A + B)' = A' \cdot B'$	DeMorgan's Theorem
$A \cdot (A + B) = A$	$A + (A \cdot B) = A$	Absorption Law
$A \cdot (A' + B) = A \cdot B$	$A + (A' \cdot B) = A + B$	
$A'' = A$		Double Negation Property
$A'B + AB' = A \oplus B$		Exclusive-Or Property
$AB + A'B' = (A \oplus B)'$		Exclusive-Nor Property

- e) Essentially, an n-bit adder is a circuit that links the circuits developed in the previous step n times, such that the carry output of the previous circuit is the carry input of the next, as illustrated below:



where the initial carry input, C_{in} , is of the Least Significant Bit (LSB) stage and the final carry output is used to determine if overflow occurred. Given the above information, what would the LSB stage be for the adder circuit?

- f) Given the relationship between twos-complement addition and subtraction, construct the R and C Boolean functions for subtraction.
g) Construct new R and C circuits that are mergers of R circuits and C circuits of the adder and subtractor with an additional control signal input (Boolean) that is used to determine both the operation and the initial carry input of an n-bit circuit configuration.

4. Hypothesis

Write your hypothesis using the format: "If [condition], then [predicted outcome], because [reasoning]."

5. Step-by-Step Procedure

- a) Define a concrete gate class for each Boolean operation required by the dual-operation circuit. Every gate class must inherit the *Gate* class.
- b) Construct the combinational sub-circuit that computes the result bit R from inputs A , B , C_{in} , and op . The leaf (input) nodes must be three *Port* objects (for A , B , C_{in}) and one *Switch* object (for op). Attach the root gate to a *Socket* object.
- c) Construct the combinational sub-circuit that computes the carry bit C from the same four inputs. Attach the root gate to a *Socket* object.
- d) Instantiate three 8-bit *Word* objects initialized to "11110000", "11001100", and "10101010", representing the A , B , and C_{in} bit streams respectively.
- e) Instantiate four 8-bit *Word* objects, each initialized to "00000000" to hold the output results for each test case.
- f) Connect the first three *Word* objects to the *Port* objects representing A , B , and C_{in} inputs for both the R and C circuits.
- g) Attach the remaining *Word* objects to the *Socket* objects connected to the root gates of both the R and C circuits.

h) Test Case 1 - Addition

Step 1: Set the *Switch* object (op) to 0 for both circuits to configure addition.

Step 2: Evaluate R and C for each bit position $i = 0, 1, \dots, 7$ using inputs A_i , B_i , and $C_{in(i)}$.

Step 3: Display and save a table showing all inputs and their computed outputs.

i) Test Case 2 - Subtraction

Step 1: Set the *Switch* object (op) to 1 for both circuits to configure subtraction.

Step 2: Evaluate R and C for each bit position $i = 0, 1, \dots, 7$.

Step 3: Display and save the results table.

- j) Construct a 4-bit ALU by chaining four single-bit dual-operation stages. Each stage's carry output feeds the next stage's carry input. The control signal *Switch* also serves as the initial carry-in for the LSB stage. Attach *Socket* objects to all result outputs except the final carry.

- k) Instantiate three 4-bit *Word* objects. Connect the first to all A *Port* inputs, the second to all B *Port* inputs, and the third to each *Socket* output object.

l) Test Case 3 - 4-bit Addition

Step 1: Assign *Word* $A = 0101$ (5) and *Word* $B = 0011$ (3).

Step 2: Set the *Switch* object to 0 for addition.

Step 3: Evaluate each *Socket* object.

Step 4: Display and save in format: $A + B = R$.

m) Test Case 4

step 1: Assign *Word* $A = 0101$ (5) and *Word* $B = 0011$ (3).

step 2: Set the *Switch* object to 1 for subtraction.

step 3: Evaluate each *Socket* object.

step 4: Display and save in format: $A - B = R$.

6. Implementation Notes

Document notable design decisions, bugs encountered, and any deviations from the planned procedure here. Include the reason for each deviation and how it affected your implementation.

7. Code Repository

List all submitted source files with a brief description of each.

8. Output

Describe and store program output for each test case in separate .txt files.

9. **Data Analysis**

Complete after running your program. Analyze trends, patterns, and anomalies with specific values.

10. **Conclusion**

Write a concise conclusion addressing the original problem statement.